UICRM 项目结构说明

项目概述

UICRM 是一个基于 STM32 的机器人控制项目，支持多种硬件平台和丰富的功能模块。项目采用模块化设计，便于扩展和维护。

项目目录结构

```
C:.
├─.github
├─.idea
├─boards
│  ├─algorithm
│  ├─base
│  │  ├─DJI_Board_TypeA_general
│  │  ├─DJI_Board_TypeC_general
│  │  ├─DM_MC01_general
│  │  ├─DM_MC02_general
│  │  └─F103_Nano_general
│  ├─components
│  ├─drivers
│  ├─platform
│  │  ├─stm32f1
│  │  ├─stm32f4
│  │  └─stm32h7
│  └─third_party
├─cmake
├─cmake-build-debug-arm-gcc-103
├─examples
│  ├─DJI_Board_TypeA_general
│  ├─DJI_Board_TypeC_general
│  ├─DM_MC01_general
│  ├─DM_MC02_general
│  └─F103_Nano_general
├─openocd
├─programs
│  ├─Hero
│  └─test
└─tools
```

1. .github

存放 GitHub 相关配置文件，如工作流定义。

2. .idea

存放 JetBrains IDE (如 CLion) 的项目配置文件。

3. boards

存放项目支持的各种硬件平台和核心组件的源代码。

3.1 algorithm

存放算法相关代码，如 PID 控制、AHRS (姿态解算)、QEKF (四元数扩展卡尔曼滤波) 等。

3.2 base

存放不同硬件平台的基础代码，包括 HAL 库、CMSIS、DSP 库、中间件 (如 FreeRTOS、USB Device) 等。

- **DJI_Board_TypeA_general**: DJI A 型开发板 (基于 STM32F427)
- **DJI_Board_TypeC_general**: DJI C 型开发板 (基于 STM32F407)
- **DM_MC01_general**: 大疆 MC01 主控板 (基于 STM32F446)
- **DM_MC02_general**: 大疆 MC02 主控板 (基于 STM32H723)
- **F103_Nano_general**: 基于 STM32F103 的 Nano 开发板

3.3 components

存放项目通用组件，如底盘控制、云台控制、摩擦轮控制、用户界面等。

3.4 drivers

存放各种外设驱动代码。

- **DJI_Board_TypeC**: DJI C 型开发板专用驱动
- **DM_MC01**: DM_MC01 主控板专用驱动
- **include**: 通用驱动头文件
- **src**: 通用驱动源文件

3.5 platform

存放平台抽象层代码，为不同芯片系列提供统一的接口。

- **stm32f1**: STM32F1 系列平台代码
- **stm32f4**: STM32F4 系列平台代码
- **stm32h7**: STM32H7 系列平台代码

3.6 third_party

存放第三方库代码，如 CRC 校验、printf 重定向、MahonyAHRS、QuaternionEKF 等。

4. cmake

存放 CMake 构建系统相关的配置文件。

5. cmake-build-debug-arm-gcc-103

存放 CMake 构建生成的文件，如编译目标、缓存等。

6. examples

存放各种示例代码，用于演示如何使用项目中的模块和功能。

- **DJI_Board_TypeA_general**: DJI A 型开发板示例
- **DJI_Board_TypeC_general**: DJI C 型开发板示例
- **DM_MC01_general**: DM_MC01 主控板示例
- **DM_MC02_general**: DM_MC02 主控板示例
- **F103_Nano_general**: F103 Nano 开发板示例

7. openocd

存放 OpenOCD 调试配置文件。

8. programs

存放完整的应用程序代码。

- **Hero**: 英雄机器人程序
- **test**: 测试程序

9. tools

存放项目相关的工具脚本。

教学建议

1. **硬件平台选择**: 根据实际使用的硬件平台，选择对应的 `boards/base` 和 `examples` 目录进行学习。
2. **模块化学习**: 从 `components` 和 `drivers` 目录入手，了解项目提供的通用功能模块和驱动。
3. **示例代码**: 通过 `examples` 目录中的示例代码，快速上手并理解如何使用项目功能。
4. **算法学习**: 深入 `boards/algorithm` 目录，学习 PID、AHRS、QEKF 等控制算法的实现。
5. **平台抽象**: 理解 `platform` 目录的作用，学习如何编写跨平台的代码。
6. **完整应用**: 学习 `programs` 目录中的完整应用程序，理解项目整体架构。